

Data Sources: <https://www.kaggle.com/datasets/radheshyamkollipara/bank-customer-churn>

```
from google.colab import files
files.upload()
```

 kaggle.json

- kaggle.json(application/json) – 74 bytes, last modified: 2025/6/20 – 100% done

Saving kaggle.json to kaggle.json

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d radheshyamkollipara/bank-customer-churn
```

 Dataset URL: <https://www.kaggle.com/datasets/radheshyamkollipara/bank-customer-churn>
 License(s): other
 Downloading bank-customer-churn.zip to /content
 0% 0.00/307k [00:00<?, ?B/s]
 100% 307k/307k [00:00<00:00, 382MB/s]

```
import zipfile
with zipfile.ZipFile("/content/bank-customer-churn.zip", "r") as bank_churn:
    bank_churn.extractall("bank-customer-churn")
```

```
import zipfile
zip_path = "bank-customer-churn.zip"
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    file_list = zip_ref.namelist()
    print("The type of ZIP profile: ")
    for file in file_list:
        print(file)
```

 The type of ZIP profile:
 Customer-Churn-Records.csv

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, roc_curve, precision_recall_curve, average_pre
from scipy.stats import multivariate_normal
from tabulate import tabulate
import io
import warnings
warnings.filterwarnings("ignore")
```

```
import os
print(os.listdir("bank-customer-churn"))
```

 ['Customer-Churn-Records.csv']

```
bank_churn = pd.read_csv("/content/bank-customer-churn/Customer-Churn-Records.csv")
bank_churn.head()
```

◆ What can I help you build?



	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	C
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	C
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1

Next steps:

[Generate code with bank_churn](#)

[View recommended plots](#)

[New interactive sheet](#)

```
bank_churn.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber             10000 non-null  int64
1   CustomerId            10000 non-null  int64
2   Surname               10000 non-null  object
3   CreditScore           10000 non-null  int64
4   Geography             10000 non-null  object
5   Gender                10000 non-null  object
6   Age                   10000 non-null  int64
7   Tenure                10000 non-null  int64
8   Balance               10000 non-null  float64
9   NumOfProducts         10000 non-null  int64
10  HasCrCard             10000 non-null  int64
11  IsActiveMember        10000 non-null  int64
12  EstimatedSalary        10000 non-null  float64
13  Exited                10000 non-null  int64
14  Complain              10000 non-null  int64
15  Satisfaction Score    10000 non-null  int64
16  Card Type             10000 non-null  object
17  Point Earned          10000 non-null  int64
dtypes: float64(2), int64(12), object(4)
memory usage: 1.4+ MB
```

```
bank_churn.describe()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	I
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	

```
bank_churn.nunique()
```



0

RowNumber	10000
CustomerId	10000
Surname	2932
CreditScore	460
Geography	3
Gender	2
Age	70
Tenure	11
Balance	6382
NumOfProducts	4
HasCrCard	2
IsActiveMember	2
EstimatedSalary	9999
Exited	2
Complain	2
Satisfaction Score	5
Card Type	4
Point Earned	785

```
import xgboost as xgb
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, auc
from sklearn.model_selection import train_test_split
y = bank_churn['Exited']
x = bank_churn.drop(columns=['Exited', 'RowNumber', 'CustomerId', 'Surname'])
x = pd.get_dummies(x, drop_first=True)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.25, random_state=42)
model = xgb.XGBClassifier(max_depth=6, n_estimators = 100, use_label_encoder=False, eval_metric = "logloss")
model.fit(x_train, y_train)
```

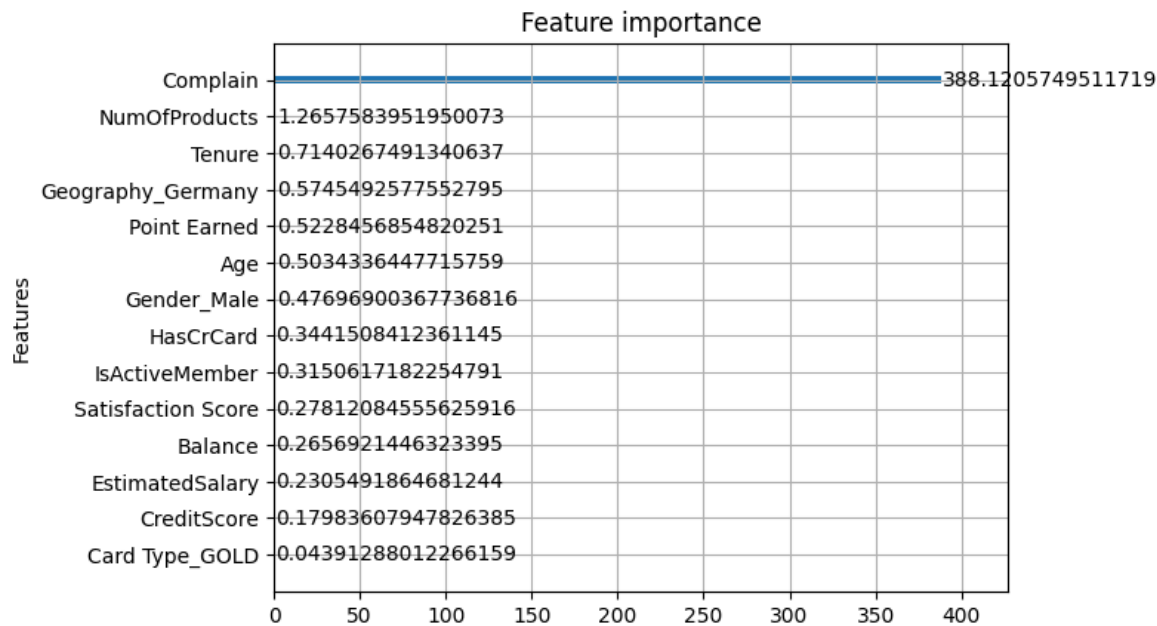


XGBClassifier



```
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               num_parallel_tree=None, random_state=None,
               subsample=None, tree_method=None)
```

```
import matplotlib.pyplot as plt
xgb.plot_importance(model, max_num_features=15, importance_type='gain')
plt.show()
```



```
import pandas as pd
importance_dict = model.get_booster().get_score(importance_type='gain')
importance_df = pd.DataFrame({
    'Feature': list(importance_dict.keys()),
    'Importance': list(importance_dict.values())
})

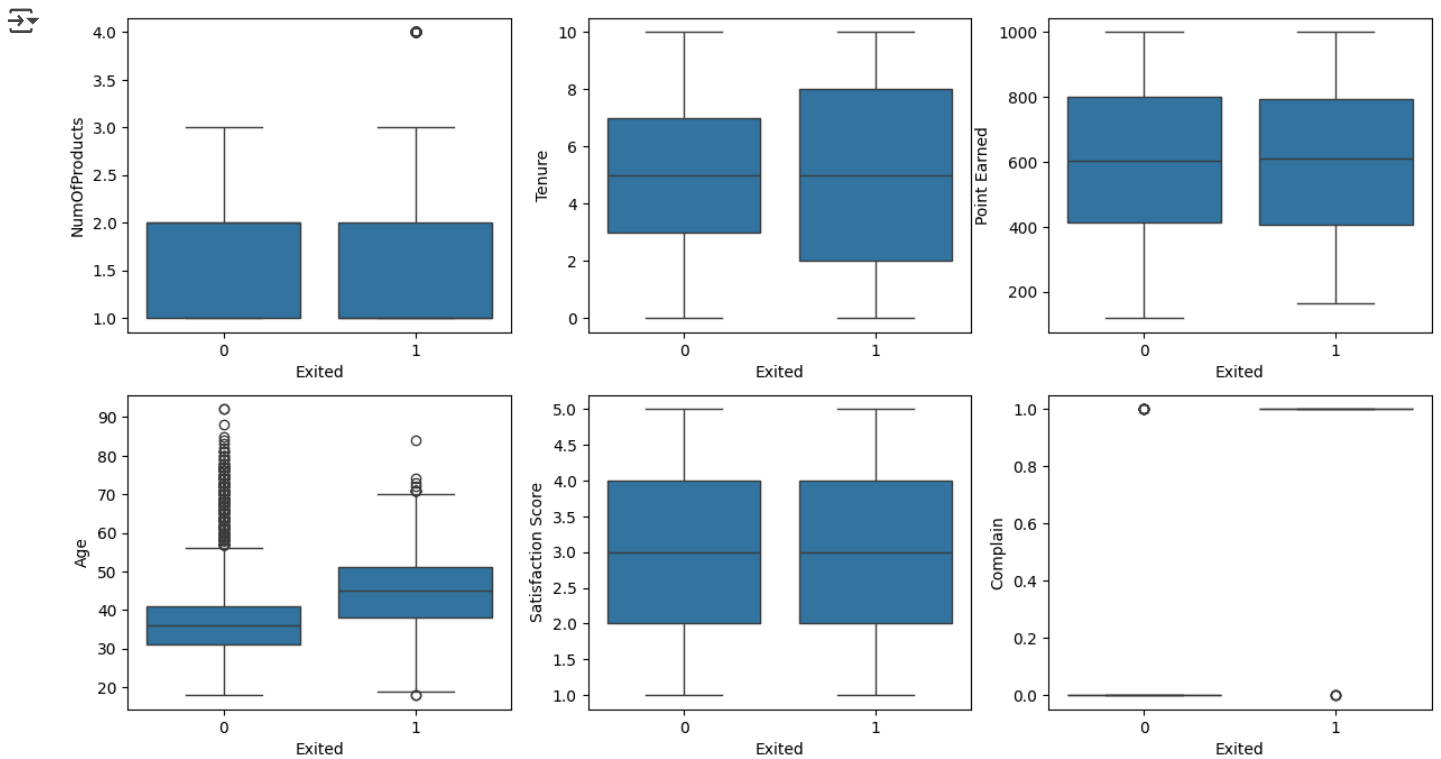
importance_df = importance_df.sort_values(by='Importance', ascending=False)
print("Top 10 important features by gain:")
print(importance_df.head(10))
```



```
Top 10 important features by gain:
   Feature  Importance
8  Complain    388.120575
4  NumOfProducts    1.265758
2    Tenure      0.714027
11 Geography_Germany    0.574549
10 Point Earned    0.522846
1    Age         0.503434
12 Gender_Male    0.476969
5    HasCrCard    0.344151
6  IsActiveMember    0.315062
9  Satisfaction Score    0.278121
```

```
import matplotlib.pyplot as plt
import seaborn as sns
features = ['NumOfProducts', 'Tenure', 'Point Earned', 'Age', 'Satisfaction Score', 'Complain']
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for i, feature in enumerate(features):
    row = i // 3
    col = i % 3
    sns.boxplot(x='Exited', y=feature, data=bank_churn, ax=axes[row, col])
    axes[row, col].set
```



```
from sklearn.preprocessing import StandardScaler
num_cols = ['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'EstimatedSalary',
            'Point Earned', 'Satisfaction Score']
scaler = StandardScaler()
scaler.fit(x_train[num_cols])
x_train[num_cols] = scaler.transform(x_train[num_cols])
x_val[num_cols] = scaler.transform(x_val[num_cols])
x_test[num_cols] = scaler.transform(x_test[num_cols])
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
classifier_logistic = LogisticRegression(max_iter=1000, random_state=42) # 增加 max_iter 防止收敛问题
classifier_KNN = KNeighborsClassifier(n_neighbors=10) # 默认5邻居, 也可调节
classifier_RF = RandomForestClassifier(n_estimators=100, random_state=42) # 常用默认值
```

```
model = xgb.XGBClassifier(max_depth=6, n_estimators=100, use_label_encoder=False, eval_metric='logloss', random_state=42)
model.fit(x_train, y_train)
```



XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=100,
              num_parallel_tree=None, random_state=42,
              ...)
```

```
y_train_pred = model.predict_proba(x_train)[:, 1]
y_val_pred = model.predict_proba(x_val)[:, 1]
```

```
train_auc = roc_auc_score(y_train, y_train_pred)
val_auc = roc_auc_score(y_val, y_val_pred)
```

```
print(f"Train AUC: {train_auc:.4f}")
print(f"Validation AUC: {val_auc:.4f}")
```

```
↗ Train AUC: 1.0000
  Validation AUC: 0.9961
```

```
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier
```

```
param_dist = {
    'max_depth': [3,5,7,9],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [100, 200, 300],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
}
```

```
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
```

```
search = RandomizedSearchCV(xgb, param_dist, n_iter=20, scoring='roc_auc', cv=5, verbose=2, random_state=42)
search.fit(x_train, y_train)
```

```
print("Best params:", search.best_params_)
print("Best AUC:", search.best_score_)
```

```
↗
```

```
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=7, n_estimators=300, subsample=0.8; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=7, n_estimators=300, subsample=0.8; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=7, n_estimators=300, subsample=0.8; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=7, n_estimators=300, subsample=0.8; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=7, n_estimators=300, subsample=0.8; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=3, n_estimators=200, subsample=1.0; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=3, n_estimators=200, subsample=1.0; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=3, n_estimators=200, subsample=1.0; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=3, n_estimators=200, subsample=1.0; total time=
[CV] END colsample_bytree=0.6, learning_rate=0.01, max_depth=3, n_estimators=200, subsample=1.0; total time=
Best params: {'subsample': 1.0, 'n_estimators': 100, 'max_depth': 9, 'learning_rate': 0.01, 'colsample_bytree': 0.6}
Best AUC: 0.999918213457358
```

```
from xgboost import XGBClassifier
best_params = search.best_params_
best_model = XGBClassifier(**best_params, use_label_encoder=False, eval_metric='logloss')
best_model.fit(x_train, y_train)
```



XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.01, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=9,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               num_parallel_tree=None, random_state=None,
               subsample=0.8, tree_method=None)
```

```
import numpy as np
x_noisy = x.copy()
x_noisy += np.random.normal(0, 0.1, x_noisy.shape)
auc_noisy = roc_auc_score(y, model.predict_proba(x_noisy)[: , 1])
print(f"AUC with noise added: {auc_noisy:.4f}")
```



AUC with noise added: 0.7669

```
from sklearn.metrics import precision_recall_curve
y_scores = model.predict_proba(x_val)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_val, y_scores)
idx = np.where(recall > 0.8)[0][-1]
print(f"Threshold: {thresholds[idx]:.4f}, Precision: {precision[idx]:.4f}, Recall: {recall[idx]:.4f}")
```



Threshold: 0.9983, Precision: 0.9940, Recall: 0.8019

```
proba = model.predict_proba(x)[: , 1]
bank_churn['Churn_Probability'] = proba
threshold = 0.95
bank_churn['Churn_Prediction'] = (bank_churn['Churn_Probability'] >= threshold).astype(int)
bank_churn['Warning'] = bank_churn['Churn_Prediction'].apply(lambda x: '⚠️' if x == 1 else '')
print(bank_churn[['Churn_Probability', 'Churn_Prediction', 'Warning']].head())
```



	Churn_Probability	Churn_Prediction	Warning
0	0.988089	1	⚠️
1	0.979219	1	⚠️
2	0.989569	1	⚠️
3	0.000352	0	
4	0.000281	0	

```
num_churn = bank_churn['Churn_Prediction'].sum()
print(f"Over threshold: {num_churn}")
```



Over threshold: 1671

```
import joblib
joblib.dump(best_model, 'best_model.pkl')
```

```
['best_model.pkl']
```

```
top_features = ['CreditScore', 'Age', 'Balance', 'EstimatedSalary', 'Tenure',
                'NumOfProducts', 'IsActiveMember', 'HasCrCard', 'Complain', 'Satisfaction Score']
```

```
!pip install scorecardpy
import scorecardpy as sc
import pandas as pd
dat_subset = dat[top_features + ['Exited']]
bins = sc.woebin(dat_subset, y='Exited', x=top_features, method='tree', no_cores=1)
woe_data = sc.woebin_ply(dat_subset, bins, no_cores=1)
x = woe_data
y = dat_subset['Exited']
```

```
Requirement already satisfied: scorecardpy in /usr/local/lib/python3.11/dist-packages (0.1.9.7)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (2.0.2)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (3.10.0)
Requirement already satisfied: scikit-learn>=0.19.1 in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (1.3.2)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (0.14.0)
Requirement already satisfied: patsy in /usr/local/lib/python3.11/dist-packages (from scorecardpy) (1.0.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.0->scorecardpy) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.0->scorecardpy) (2023.3)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.0->scorecardpy) (2023.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.19.1->scorecardpy) (1.10.0)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.19.1->scorecardpy) (1.3.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.19.1->scorecardpy) (3.2.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (1.0.7)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (23.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (9.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->scorecardpy) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->scorecardpy) (1.16.0)
[INFO] creating woe binning ...
Binning on 10000 rows and 11 columns in 00:00:10
[INFO] converting into woe values ...
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
lr = LogisticRegression(max_iter=1000)
lr.fit(x_train, y_train)
print("Train accuracy:", lr.score(x_train, y_train))
print("Test accuracy:", lr.score(x_test, y_test))
```

```
Train accuracy: 0.9991428571428571
Test accuracy: 0.9996666666666667
```

```
card = sc.scorecard(bins, lr, X.columns.tolist(), points0=600, odds0=1/99, pdo=30)
```

```
score = sc.scorecard_ply(dat_subset, card, only_total_score=True)
print(score.head())
```

```
score
0 368.0
1 399.0
2 361.0
3 750.0
4 724.0
```

```
def rating(score):
    if score >= 750:
```



```

    return 'AAA'
elif score >= 700:
    return 'AA'
elif score >= 650:
    return 'A'
elif score >= 600:
    return 'BBB'
else:
    return 'BB or below'

```

给分数打评级

```

score['Rating'] = score['score'].apply(rating)
print(score.head())

```

```

↗ score      Rating
0  368.0  BB or below
1  399.0  BB or below
2  361.0  BB or below
3  750.0      AAA
4  724.0      AA

```

```

rating_order = ['AAA', 'AA', 'A', 'BBB', 'BB or below']
rating_counts = score['Rating'].value_counts().reindex(rating_order).fillna(0).astype(int)
print(rating_counts)

```

```

↗ Rating
AAA      5412
AA       1710

```